

Rewind project report draft

Practice Module for Certificate in Designing Modern Software Systems

Team number pending confirmation

Members:

Gopal Guruprasath · Nguyen Kim Long · Sang Haoxiang

Phan Mai Tan Loi · Jiang Jiayu

CONTENTS

1. INTRODUCTION.....	3
1.1 Background.....	3
1.2 Business Needs.....	3
1.3 Stakeholders.....	3
1.4 Project Scope.....	3
1.4.1 Functionality in Scope.....	3
1.4.2 Functionality out of Scope.....	3
1.4.3 Quality Attributes.....	4
2. PROJECT CONDUCT.....	5
2.1 Project Plan.....	5
2.2 Project Status.....	5
2.3 Project Metrics.....	5
3. SYSTEM DESIGN.....	6
3.1 Software Architecture.....	6
3.2 Transition from Analysis to Design.....	7
3.2.1 Static and dynamic transition guidance.....	8
3.3 Use Case Model.....	9
3.3.1 Overall use case diagram.....	9
3.3.2 Use case descriptions.....	9
3.4 Analysis and Design Models.....	10
3.4.1 Package models and ownership.....	10
3.5 Design Problems and Patterns.....	10
3.5.1 Five code backed design problems.....	10
3.6 Database Schemas.....	11
4. DEVSECOPS AND DEVELOPMENT LIFECYCLE.....	12
4.1 Source Control Strategy.....	12
4.2 Continuous Integration.....	12
4.3 Continuous Delivery.....	12
5. Individual members activity contribution summary.....	13

1. Introduction

1.1 Background

Rewind is a private shared-memory application for close friend groups whose members may live apart. Members contribute short ordinary moments to a time capsule. Submissions become sealed metadata; a server-driven cycle later compiles and reveals a chronological film, which remains in the group Archive. The submitted proposal frames delayed reveal as a way to reduce pressure to curate every contribution. That expected benefit has not been established by a measured user study in this draft.

1.2 Business Needs

The system coordinates trusted membership, bounded contributions, timed reveal and reliable media delivery. A group needs confidence that only members can access its conversation and released film, that quotas and corrections remain consistent across devices, and that an interrupted process can recover without duplicate contributions or lost accepted media. These needs drive explicit authorization, transactional rules, durable jobs and honest client states.

1.3 Stakeholders

The five proposal members are Gopal Guruprasath, Nguyen Kim Long, Sang Haoxiang, Phan Mai Tan Loi and Jiang Jiayu. Their individual model ownership, contributions and effort require factual confirmation. Other stakeholders are private-group owners and invited members, authorized operators and the module assessors. There is no recorded commercial sponsor or lecturer approval of the pilot exceptions.

1.4 Project Scope

Sprint 2 develops a usable private-group web journey. The current release clients are desktop web, iPhone Safari and the installed Home Screen web app. The codebase also contains Expo/native adapters; Android/APK and compiled iOS obligations remain visible but are deferred for this overnight goal. The final proposal still requires managed OIDC, PostgreSQL, recoverable managed backups and retro processing before upload in Sprint 3. A working password/SQLite/server-processing pilot does not satisfy those final obligations.

1.4.1 Functionality in Scope

UC01–UC03 cover account/session, private group and invitation access. UC04–UC06 cover video review/trim/submission, photo contributions and quota/correction. UC07–UC08 cover automatic cycle rollover, compilation/retry/publication and archival. UC09–UC11 cover owner prompt/timezone, opt-in reminders and private text/reply/reaction/reconnect. UC12–UC14 cover premiere/Archive playback, own-clip/group-film download and client install/update/invite delivery. Five packages organize these fourteen use cases; they do not cap the number of major-flow diagrams.

1.4.2 Functionality out of Scope

The first delivery excludes public feeds/profiles/discovery, followers, hashtags, licensed music, owner transfer, member removal, leaving/rejoining and account

deletion/recovery. Chat editing/deletion, read receipts, typing and attachments are excluded. Public-store approval and a commercial SLA are not promised. The membership exclusions are privacy/retention limitations, not complete production account management.

1.4.3 Quality Attributes

The proposal requires server-enforced privacy, least privilege, authenticated HTTPS, idempotent writes, source-audio preservation, valid portrait video, bounded retries, clear failure states and repeatable delivery. Its performance targets include API p95 at most two seconds and a 25-clip/150-second compilation workload within ten minutes. Measured results must name build, fixture, hardware and controls; green tests alone are not those measurements. Real storage semantics, provider receipt, physical camera and installed-client lifecycle evidence remain distinct.

2. Project Conduct

2.1 Project Plan

Sprint names are zero-based: Sprint 0 foundation; Sprint 1 13–26 September; Sprint 2 27 September–10 October; Sprint 3 11–24 October 2026. The owner execution plan estimates 211–365 focused hours for Sprint 2 and 76–132 for the deferred compliance tranche. These are planning estimates, not recorded effort. The WBS groups entry/access, capture/media, cycle/compilation, participation/reminders, authorized delivery, operations and five report packages. Actual member responsibility for at least one use case and one pattern problem is pending; agent task ownership is not human contribution evidence.

2.2 Project Status

Accepted dev contains the local-password/SQLite pilot, private groups/invites, capture review, contribution policy, persisted chat, cycle processing, authorized Archive delivery and the public-only PWA shell. PR #387 repairs #384's photo group label. PR #388 repairs chat #385's API prefix; two local HTTPS two-member UI journeys passed retry and retained history. #251/#252 retain broader acceptance. Safari/Home Screen #329/#350, physical orientation #327, hosted/S3 #344 and installed playback/download #345 remain open at their unmet gates. #352 is an aggregate journey outcome, not an extra implementation closure.

2.3 Project Metrics

The fresh baseline was fc2b8c811e9c13b176113fa0bdb2c015798990b4 with successful Quality and CodeQL runs. PR #387 tested head 64c6e40e56da31ab602bd6c82384b5d9241b7251 and merged as f1d74c510ecff566a25b1449daa8352befa9f00c with an identical Git tree. Its fast checks passed 148 root tests with one existing skip, 443 server tests and 605 frontend tests. These counts describe that exact run; they are not human productivity or time spent. Actual Sprint meeting decisions, burndown data and per-member hours remain pending where no verified record supplies them.

3. System Design

3.1 Software Architecture

Current pilot architecture

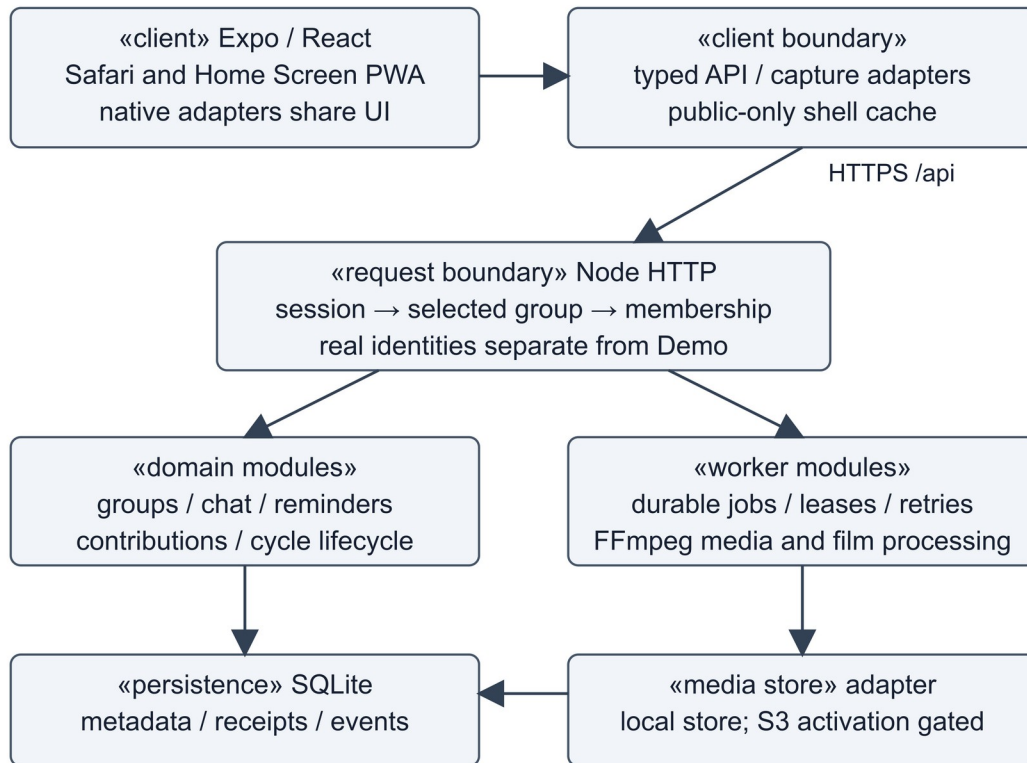


Figure 1 Current component and layer model. Source: accepted dev f1d74c5; src/runtime, server/src, server/migrations.

Future compliance target

Sprint 3 obligations; not current acceptance

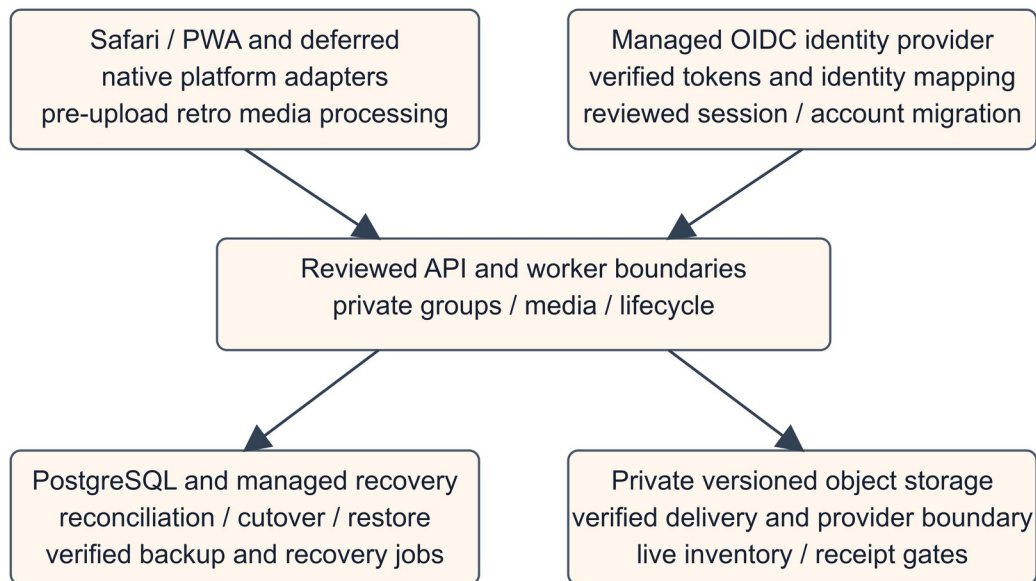


Figure 2 Final compliance target. OIDC, PostgreSQL, managed recovery and pre-upload retro remain Sprint 3 obligations.

The current implementation uses a TypeScript Expo/React client, React Native web/PWA output, a modular Node HTTP service, SQLite metadata and FFmpeg media processing. The web shell and API share an HTTPS origin with an /api proxy boundary. Browser authority uses an HttpOnly cookie; native adapters use a securely stored opaque token. Real and synthetic Demo identities are separate. Public shell caching never authorizes or caches private API/media responses.

Capture, group, chat, reminder and Archive UI modules call typed adapters. The server establishes the principal, selected group and membership before domain operations. SQLite transactions persist quota, invitation, contribution and lifecycle changes. Durable jobs/leases coordinate processing and retries; released-media capabilities recheck session, selected group, release and exact media integrity. Private S3/direct-upload code and isolated Terraform roots are prepared; actual hosted S3 semantics and live inventory remain separate gates. This source description does not assert every prepared adapter is activated in the hosted pilot.

The final target replaces pilot password verification with managed OIDC and SQLite with reviewed PostgreSQL, adds managed recovery, and processes retro treatment before upload. Provider notifications, final native packaging and live storage acceptance require their own verified boundaries. Target architecture is labelled separately from current implementation.

3.2 Transition from Analysis to Design

Six strategies guide the complete journey. T1 trusted identity maps conceptual Account/Member to session resolution, request-boundary adapters and transaction-

bound membership predicates. T2 platform capture maps a conceptual Recording to camera/file/store/review contracts and explicit ownership/disposal. T3 transactional contribution maps Budget/Correction to durable contribution/job/quota records and idempotency keys. T4 persisted lifecycle maps Cycle/Film to recorded state transitions, successors, leases and retryable jobs. T5 asynchronous participation maps Conversation/Reminder to ordered events, reconnect cursors, per-session destinations and durable outbox/provider adapters. T6 authorized delivery maps ReleasedMedia to fresh session/group-bound capabilities, client players/download adapters and public-only asset caching.

3.2.1 Static and dynamic transition guidance

For T1, keep principals distinct from client resource IDs; validate session before membership/owner query, then mutate within the authorized transaction. For T2, create a platform adapter and owned local media handle; review precedes submission and retake/success disposes content. For T3, bind a retry key to one contribution and apply both quota dimensions atomically. For T4, persist cycle closure/successor and durable work before execution; publication must observe ready output and maintain the 24-hour premiere. For T5, separate durable facts from live transport; reconnect uses a watermark and notification delivery is not inferred from scheduling. For T6, refresh temporary access at each play/download and clear stale protected media on group/session change. The package model pairs show these concrete static/dynamic mappings; complete final diagram incorporation remains part of #362.

3.3 Use Case Model

3.3.1 Overall use case diagram

Overall use case model

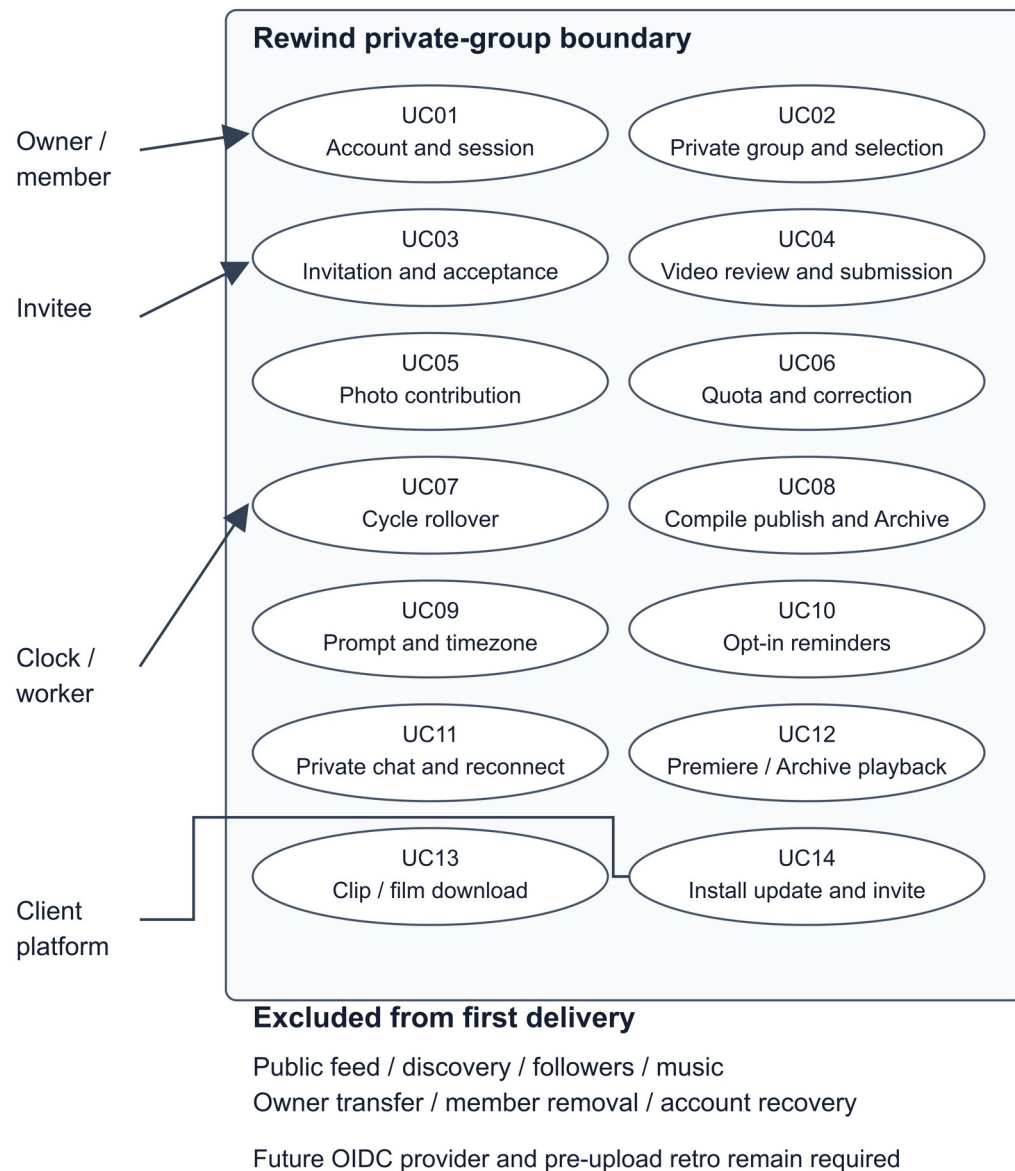


Figure 3 Overall use case catalogue. Actor associations are an overview; package flows refine the authorization roles.

The overall model groups UC01–UC14 under the private Rewind boundary, with owner/member, invitee, clock/worker and platform roles. Excluded public-social and account-management cases are outside the boundary. Managed OIDC is a future provider role. Package-specific diagrams refine the same boundary.

3.3.2 Use case descriptions

Each package's use-cases.md declares normal and relevant exceptional major flows: access rejection/expiry/replay; unsupported capture/permission, invalid media,

upload interruption; quota and correction boundaries; scheduler/processing failure and restart; chat reconnect/denial and reminder expiry; sealed/unavailable/offline media and client updates. A flow catalogue must enumerate every required paired sequence; a single happy-path sequence does not cover all major flows.

3.4 Analysis and Design Models

3.4.1 Package models and ownership

ACCESS UC01–UC03, CAPTURE UC04–UC06, CYCLE UC07–UC08, PARTICIPATION UC09–UC11 and ARCHIVE UC12–UC14 are separate source packages. Analysis class views describe domain responsibilities; design views name implemented modules/interfaces/persistence. Each major flow requires one analysis and one design sequence. The bounded draft links those editable sources and rendered figures; embedding every final model, completing the cross-package coverage audit and assigning real human ownership remain pending. No member is assigned a use case from commit counts.

Detailed model sources: ACCESS, CAPTURE, CYCLE, PARTICIPATION, ARCHIVE.

3.5 Design Problems and Patterns

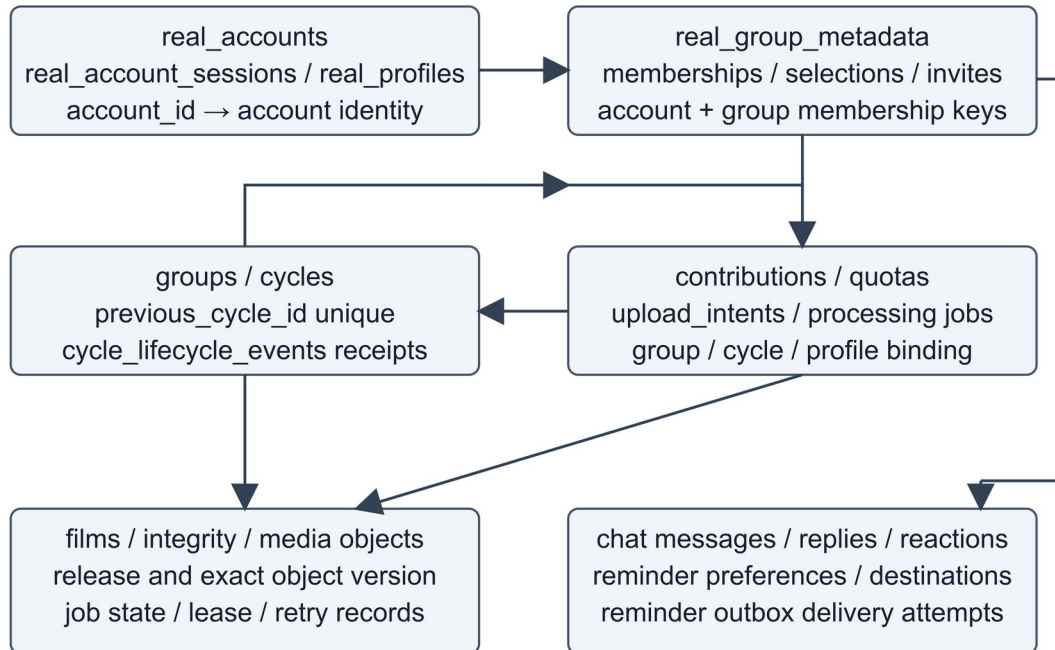
3.5.1 Five code backed design problems

ACCESS compares Policy/Strategy with Chain of Responsibility for mixed identity and membership boundaries; actual implementation uses explicit adapters and function policy, with real-account predicates distinct from Demo. CAPTURE compares Adapter/Strategy/Template Method for platform/file/upload/processing variation. CYCLE compares State/Command/polling against functional persisted transitions and durable queue/leases. PARTICIPATION compares Observer/Adapter/Strategy for events, reconnect and provider/outbox coordination. ARCHIVE compares Proxy/Adapter/Facade for private capabilities and portable player/download delivery. Before figures are explicit design alternatives, not invented historical vulnerabilities. Current function-centered code must not be described as an unimplemented pattern class hierarchy. The package discussions supply candidate rationale, before/after class and sequence views and exact source pins; human pattern attribution and final implementation refresh remain pending.

3.6 Database Schemas

Current SQLite logical schema

Selected relation clusters; full DDL is in server/migrations



Demo identity records remain separate from real accounts.

PostgreSQL parity and managed restore are future obligations.

Figure 4 Selected logical SQLite relations. Full foreign keys and migration DDL remain the authoritative physical schema.

The current schema is SQLite through versioned migrations. Real accounts/sessions/profiles, group metadata/memberships/selections/invites stay separate from Demo identities. Group/cycle/contribution/quota/correction records connect to persisted media jobs, lifecycle events, films, chat/events and reminder preferences/destinations/outbox. Upload intents bind account/profile/group/cycle to exact object versions and final contribution/jobs. Foreign keys, unique retry identities and transaction boundaries enforce these relations. The current schema diagrams are selected logical views, with full DDL in server/migrations. PostgreSQL parity, data reconciliation, controlled cutover and managed restore are future #175/#261/#172 obligations; no live PostgreSQL schema or restore is claimed.

4. DevSecOps and Development Lifecycle

4.1 Source Control Strategy

rewind-app is the authoritative repository for code, backlog and planning. Issue branches start from refreshed dev under guru/ and focused PRs target dev with Refs references. Routine dev integration requires up-to-date green aggregate Quality and resolved conversations; auth/private-media/migration/deployment/infra changes need human review. main remains the reviewed release branch. A merge is not deployment or user acceptance. Dirty primary Terraform/planning work and archived workspace boundaries remain protected. GitHub access in this run uses authenticated gh exclusively.

4.2 Continuous Integration

Quality classifies the exact diff and runs relevant format, lint, architecture, typecheck, production audit, root/infra, server/coverage, frontend/coverage, web/browser and cloud-free deployment lanes. The aggregate required check reports each lane truthfully; docs-only or unchanged browser lanes may be skipped by the accepted scope rules. Frontend statements have a 70 percent gate; server coverage is measured separately without a percentage gate. CodeQL, GitGuardian and release image/IaC scans supply distinct security evidence. Pending or absent scans are not a clean result. Local focused/fast checks precede relevant slower browser/media verification.

4.3 Continuous Delivery

The accepted Deploy dev workflow runs for dev pushes or manual dispatch, classifies docs-only changes, requires successful dev Quality and uses the existing dev environment/OIDC deployment role to verify a release bundle, activate it after health checks and restore the prior release on failure. This explains accepted automation; this overnight goal does not provision or apply infrastructure. One existing hosted pilot path is evidenced, while two isolated dev/release roots and private storage configuration are prepared but still require reviewed live plans/inventory/activation. main promotion, actual deployment, hosted acceptance and final release approval remain distinct.

5. Individual members activity contribution summary

The proposal names the five members listed in §1.3. For each, the final report must record actual work, evidence, owned use case, owned design problem and rough effort supplied or confirmed by that person. These fields are pending. Jiayu and Long's actual research/review comments under #267 may be cited as attributed observations; the closed unmerged #325 revision is not claimed as accepted code or document content. Agent-generated model drafts and automated test counts do not establish a member's hours, design judgment or lecturer approval.

References and final draft gates

Issue #362 owns final assembly. Source packages are under packages/access, capture, cycle, participation and archive. The submitted proposal and report template are named retained sources; their SHA-256 identities are recorded in template-contract.md. Required final gates are accepted packages and full diagram/link audit, factual member ownership/effort, actual Scrum/review records, final OIDC/PostgreSQL/backups/pre-upload retro, relevant hosted/device/provider acceptance, and final assessment sign-off. The draft preserves these outcomes without claiming completion.

Template fidelity: the supplied A4 layout, named styles, numbering, header logo and footer fields are preserved. Native contents/page fields were refreshed in Microsoft Word and verified in the rendered draft. This export is a draft, not final assessment acceptance.